

Dynamic Portfolio System

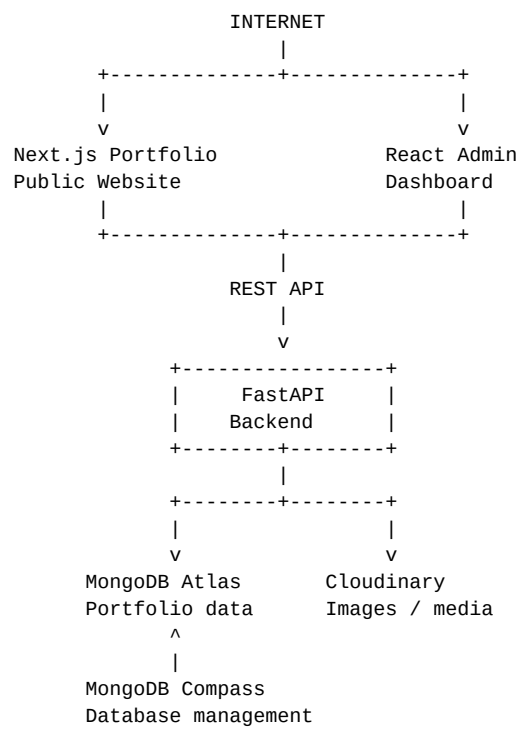
Architecture & Backend Development Specification

A production-oriented architecture for converting the existing portfolio into a dynamic CMS-driven portfolio with a React admin panel, FastAPI backend, MongoDB Atlas database, Cloudinary media storage, and an SEO-focused Next.js public website.

1. Technology Stack

Layer	Technology	Purpose
Public website	Next.js	SEO-friendly dynamic portfolio
Admin panel	React	Authenticated content management
Backend	FastAPI / Python	REST API, validation, business logic, authentication
Database	MongoDB Atlas	Persistent portfolio data
Database GUI	MongoDB Compass	Inspect and manage MongoDB data
Media	Cloudinary	Image/media upload, storage and delivery
Backend hosting	Render	Deploy FastAPI
Frontend hosting	Vercel / Netlify	Deploy Next.js and React admin

2. High-Level Architecture



3. Project Structure

```
dipesh-portfolio-system/
|
```

```

+-- portfolio-backend/
+-- portfolio-admin/
+-- portfolio-web/

```

4. FastAPI Backend Architecture

```

portfolio-backend/
|
+-- app/
|   +-- main.py
|   +-- core/
|       +-- config.py
|       +-- security.py
|       +-- database.py
|       +-- cloundinary.py
|       |
|       +-- models/
|           +-- admin.py
|           +-- profile.py
|           +-- experience.py
|           +-- education.py
|           +-- project.py
|           +-- skill.py
|           +-- certification.py
|           +-- achievement.py
|           +-- social.py
|           +-- contact.py
|           +-- site_settings.py
|           |
|       +-- schemas/
|       +-- repositories/
|       +-- services/
|           +-- auth_service.py
|           +-- profile_service.py
|           +-- project_service.py
|           +-- experience_service.py
|           +-- media_service.py
|           |
|       +-- routers/
|           +-- auth.py
|           +-- public.py
|           +-- profile.py
|           +-- projects.py
|           +-- experience.py
|           +-- education.py
|           +-- skills.py
|           +-- certifications.py
|           +-- achievements.py
|           +-- socials.py
|           +-- settings.py
|           +-- media.py
|
+-- tests/
+-- .env
+-- .env.example
+-- requirements.txt
+-- Dockerfile
+-- README.md

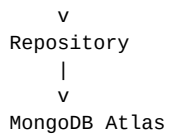
```

5. Layered Backend Flow

```

HTTP Request
|
v
Router
|
v
Schema validation
|
v
Service / business logic
|

```



For media:
Router -> Media Service -> Cloudinary -> MongoDB reference

6. MongoDB Collections

- admins
- profile
- experiences
- education
- projects
- skills
- certifications
- achievements
- social_links
- site_settings
- contact_messages

Possible future collections: blog_posts, categories, testimonials, services, resume_versions. These should only be added when required.

7. Core Data Models

Profile

```
{
  "name": "Dipesh Kumar Mandal",
  "headline": "Software Engineer",
  "short_bio": "...",
  "about": "...",
  "profile_image": { "url": "...", "public_id": "..." },
  "location": "Kathmandu, Nepal",
  "email": "...",
  "phone": "...",
  "resume_url": "...",
  "availability": true
}
```

Experience

```
{
  "company": "...",
  "position": "...",
  "employment_type": "Full-time",
  "location": "Remote",
  "start_date": "YYYY-MM",
  "end_date": null,
  "is_current": true,
  "description": "...",
  "technologies": ["React Native", "Next.js"],
  "company_url": "...",
  "display_order": 1
}
```

Education

```
{
  "institution": "...",
  "degree": "...",
  "field": "...",
  "start_date": "...",
  "end_date": "...",
  "description": "...",
  "grade": "...",
  "location": "...",
  "display_order": 1
}
```

Project

```
{
  "title": "Diet Food Recommendation System",
  "slug": "diet-food-recommendation-system",
  "short_description": "...",
  "description": "...",
  "thumbnail": { "url": "...", "public_id": "..." },
  "images": [],
  "technologies": ["Python", "FastAPI", "MongoDB"],
  "github_url": "...",
  "live_url": "...",
  "featured": true,
  "status": "completed",
  "display_order": 1
}
```

Skill

```
{
  "name": "React Native",
  "category": "Mobile Development",
  "level": "Advanced",
  "icon": "react",
  "display_order": 1
}
```

```
}
```

Certification

```
{
  "title": "...",
  "issuer": "...",
  "issue_date": "...",
  "credential_id": "...",
  "credential_url": "...",
  "certificate_image": { "url": "...", "public_id": "..." }
}
```

Achievement

```
{
  "title": "...",
  "description": "...",
  "date": "...",
  "url": "...",
  "display_order": 1
}
```

Social Link

```
{
  "platform": "GitHub",
  "url": "...",
  "icon": "github",
  "display_order": 1,
  "enabled": true
}
```

8. Common Media Object

```
{
  "url": "https://res.cloudinary.com/...",
  "public_id": "portfolio/projects/project-name",
  "alt": "Descriptive alternative text",
  "width": 1200,
  "height": 800
}
```

Actual image files live in Cloudinary. MongoDB stores the Cloudinary URL and public_id plus useful metadata. This avoids storing binary images directly in MongoDB.

9. API Design

```
/api/v1/
|
+-- auth/
|   +-- POST /login
|
+-- admin/
|   +-- profile
|   +-- experiences
|   +-- education
|   +-- projects
|   +-- skills
|   +-- certifications
|   +-- achievements
|   +-- social-links
|   +-- settings
|
+-- public/
|   +-- portfolio
|   +-- profile
|   +-- experiences
|   +-- education
|   +-- projects
```

```
|   +-- skills
|   +-- certifications
|   +-- achievements
|   +-- socials
|   +-- settings
|
+-- media/
    +-- upload
    +-- delete
```

10. Public Portfolio Endpoint

GET /api/v1/public/portfolio

Response:

```
{
  "profile": {},
  "experiences": [],
  "education": [],
  "projects": [],
  "skills": [],
  "certifications": [],
  "achievements": [],
  "social_links": [],
  "settings": {}
}
```

The combined endpoint minimizes requests from the public portfolio while individual endpoints remain available when needed.

11. Authentication

```
React Admin
  |
  | POST /api/v1/auth/login
  v
FastAPI
  |
  v
Validate admin credentials
  |
  v
JWT access token
  |
  v
Authenticated admin requests

Protected endpoints:
GET/POST/PUT/DELETE /api/v1/admin/...
```

Authentication should include password hashing, JWT-based authorization, protected admin routes, environment-based secrets, and consistent authentication dependencies.

12. Cloudinary Upload Flow

```
React Admin
  |
  | image/file
  v
FastAPI media endpoint
  |
  v
Cloudinary
  |
  +--> secure URL
  +--> public_id
  |
  v
MongoDB document stores media reference
```

When replacing an image, the backend should delete the old Cloudinary asset when appropriate, upload the new asset, and then update MongoDB. When deleting an entity, associated media should also be cleaned up to prevent orphaned assets.

13. Next.js Public Website

```
portfolio-web/
|
+-- app/
|   +-- layout.tsx
|   +-- page.tsx
|   +-- projects/
|       |   +-- page.tsx
|       |   +-- [slug]/page.tsx
|   +-- experience/page.tsx
|   +-- resume/page.tsx
|   +-- contact/page.tsx
|   +-- robots.ts
|   +-- sitemap.ts
|   +-- opengraph-image.tsx
|
+-- components/
|   +-- layout/
|   +-- sections/
|   +-- ui/
|   +-- projects/
|
+-- lib/
|   +-- api.ts
```

```
|   +-- portfolio.ts
|   +-- types.ts
```

14. SEO Requirements

- Dynamic page title and description
- Canonical URLs
- Open Graph metadata
- Social sharing metadata
- robots.txt
- sitemap.xml
- JSON-LD structured data
- Semantic HTML
- Correct H1/H2 hierarchy
- SEO-friendly project slugs
- Optimized images and alt text
- Good Core Web Vitals / page performance

Example project URL

/projects/diet-food-recommendation-system

15. React Admin Panel

```
portfolio-admin/
|
+-- src/
|   +-- components/
|   |   +-- ui/
|   |   +-- forms/
|   |   +-- tables/
|   |   +-- layout/
|   |
|   +-- pages/
|   |   +-- Login/
|   |   +-- Dashboard/
|   |   +-- Profile/
|   |   +-- Experience/
|   |   +-- Education/
|   |   +-- Projects/
|   |   +-- Skills/
|   |   +-- Certifications/
|   |   +-- Achievements/
|   |   +-- SocialLinks/
|   |   +-- Settings/
|   |
|   +-- services/api.ts
|   +-- hooks/
|   +-- context/AuthContext.tsx
|   +-- types/
|   +-- routes/
```

Admin capabilities

- Login/logout
- Dashboard overview
- Create, edit, delete and reorder portfolio content

- Upload and replace images
- Manage profile and CV information
- Manage projects and technologies
- Manage SEO/site settings
- View contact messages
- Enable/disable social links

16. Contact Form Flow

```
Visitor
|
v
Next.js Contact Form
|
v
FastAPI
|
v
MongoDB contact_messages
|
v
React Admin -> Messages
```

17. Deployment Architecture

```
Next.js Portfolio  -----> Vercel / Netlify
React Admin       -----> Vercel / Netlify
FastAPI Backend   -----> Render
MongoDB           -----> MongoDB Atlas
Images            -----> Cloudinary
Database GUI      -----> MongoDB Compass
```

18. Environment Variables

```
# Backend
MONGODB_URI=
MONGODB_DATABASE=
JWT_SECRET=
JWT_ALGORITHM=
ACCESS_TOKEN_EXPIRE_MINUTES=

CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

FRONTEND_URL=
ADMIN_URL=

# Never commit .env to Git.
```

19. Development Roadmap

Phase 1 — Architecture

Finalize requirements, collections, schemas, API contracts, authentication and folder structure.

Phase 2 — Backend

Create FastAPI project, virtual environment, dependencies, configuration, MongoDB connection, models, schemas, repositories, services, routers, authentication, CRUD, public endpoint and tests.

Phase 3 — Admin

Build React admin, authentication, dashboard, CRUD forms, tables, settings, media upload and contact messages.

Phase 4 — Portfolio

Build Next.js layout, API integration, hero, about, experience, education, skills, projects, certifications, contact and resume.

Phase 5 — SEO

Metadata, dynamic metadata, sitemap, robots, Open Graph, JSON-LD, canonical URLs, semantic HTML and performance.

Phase 6 — Deployment

MongoDB Atlas, Render, frontend deployment, environment variables, CORS, production testing and custom domain.

20. Backend Development Principles

- Keep routers thin; business logic belongs in services.
- Use Pydantic schemas for request and response validation.
- Keep database access inside repositories.
- Keep secrets only in environment variables.
- Use versioned API routes under `/api/v1`.
- Separate public endpoints from protected admin endpoints.
- Return predictable response structures and errors.
- Validate uploaded files and restrict acceptable media types.
- Clean up Cloudinary assets when records are deleted or replaced.
- Write tests for authentication, CRUD, validation and public portfolio aggregation.
- Keep the API independent from the UI so both React Admin and Next.js can consume it.

21. Backend Starting Point

We will start implementation with the backend rather than building the admin and public website at the same time. The first milestone is a clean FastAPI foundation with configuration, MongoDB Atlas connection, Cloudinary configuration, health check, API versioning and a documented project structure. After that, we will implement authentication and the portfolio domain models one module at a time.

22. Definition of Done for Backend MVP

- FastAPI runs locally and on Render.
- MongoDB Atlas connection works securely.
- Cloudinary upload/delete works through the backend.
- Admin authentication works.
- All core portfolio collections have CRUD APIs.
- Public portfolio endpoint returns complete portfolio data.
- Validation and error handling are consistent.
- CORS is configured for the deployed frontends.
- Environment variables are documented.
- Automated tests cover the important backend flows.
- Swagger/OpenAPI documentation is usable for frontend integration.